torch

CSCI 270

torch is a python package for computing with tensors

1.8

$$\begin{bmatrix} 20 \\ 28 \\ 41 \end{bmatrix}$$

$$\begin{bmatrix} .72 & .06 \\ .34 & .25 \\ .17 & .57 \end{bmatrix}$$

scalar vector matrix and beyond

0 — 1 — 2 — 3+ →

dimension

you can initialize tensors from python lists

```
In [1]: from torch import tensor
In [2]: a = tensor(1.8)
In [3]: b = tensor([20, 28, 41])
In [4]: c = tensor([[.72, .06], [.34, .25], [.17, .57]])
In [5]: d = tensor([[[1, 2, 3], [4, 5, 6], [7, 8, 9]],
   ...:             [[10, 11, 12], [13, 14, 15], [16, 17, 18]],
   ...:             [[19, 20, 21], [22, 23, 24], [25, 26, 27]]])
```

scalar

1.8

vector

$$\begin{bmatrix} 20 \\ 28 \\ 41 \end{bmatrix}$$

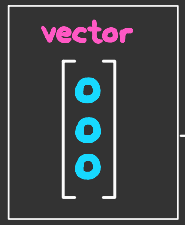
matrix

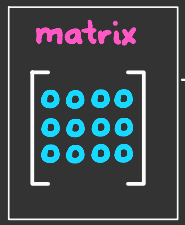
$$\begin{bmatrix} .72 & .06 \\ .34 & .25 \\ .17 & .57 \end{bmatrix}$$

higher order tensor

```
In [6]: a
Out[6]: tensor(1.8000)

In [7]: b
Out[7]: tensor([20, 28, 41])

In [8]: c
Out[8]:
tensor([[0.7200, 0.0600],
        [0.3400, 0.2500],
        [0.1700, 0.5700]])

In [9]: d
Out[9]:
tensor([[[ 1,  2,  3],
         [ 4,  5,  6],
         [ 7,  8,  9]],

        [[10, 11, 12],
         [13, 14, 15],
         [16, 17, 18]],

        [[19, 20, 21],
         [22, 23, 24],
         [25, 26, 27]]])
```

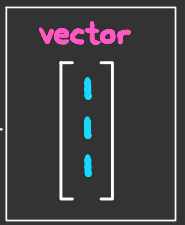
or by shape

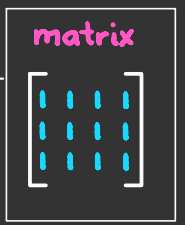





```
In [1]: import torch

In [2]: torch.zeros(3)
Out[2]: tensor([0., 0., 0.])

In [3]: torch.zeros(3, 4)
Out[3]:
tensor([[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]])

In [4]: torch.zeros(2, 3, 4)
Out[4]:
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],

        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]]])
```

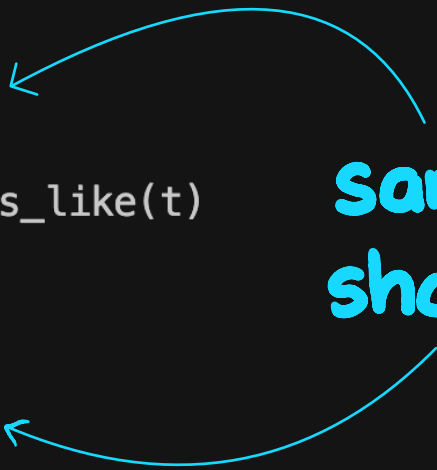
```
In [1]: import torch

In [2]: torch.ones(3)
Out[2]: tensor([1., 1., 1.])

In [3]: torch.ones(3, 4)
Out[3]:
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])

In [4]: torch.ones(2, 3, 4)
Out[4]:
tensor([[[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]],

        [[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]]])
```



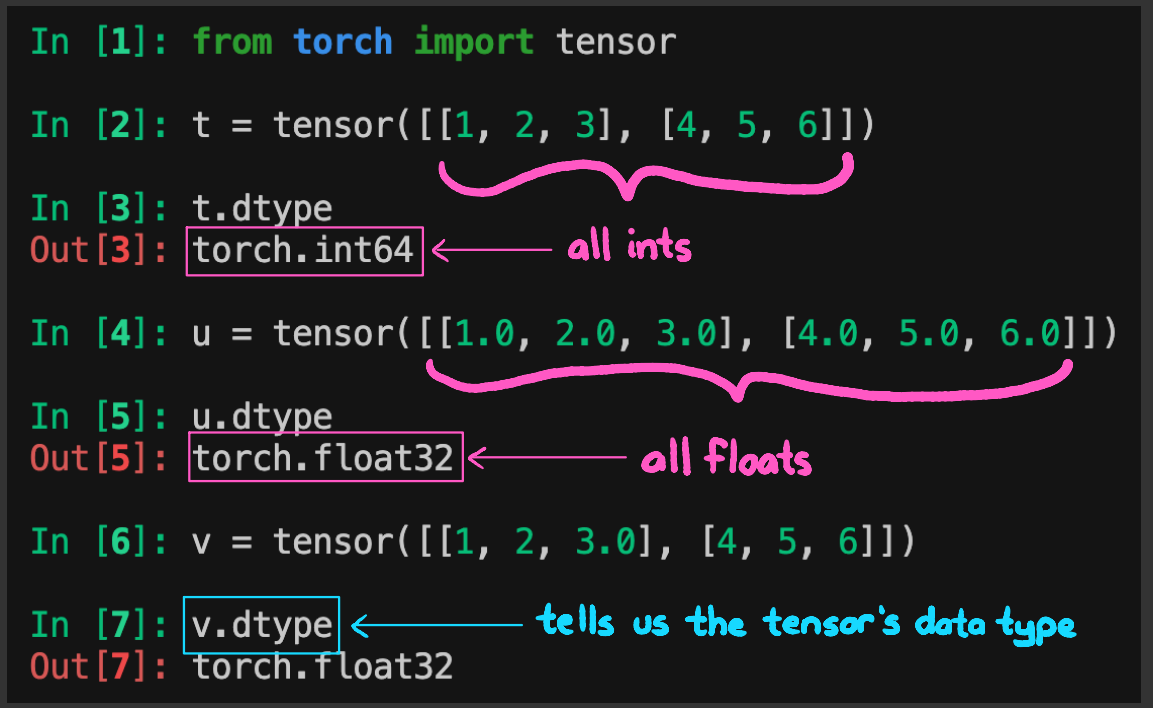


or by emulating other tensors

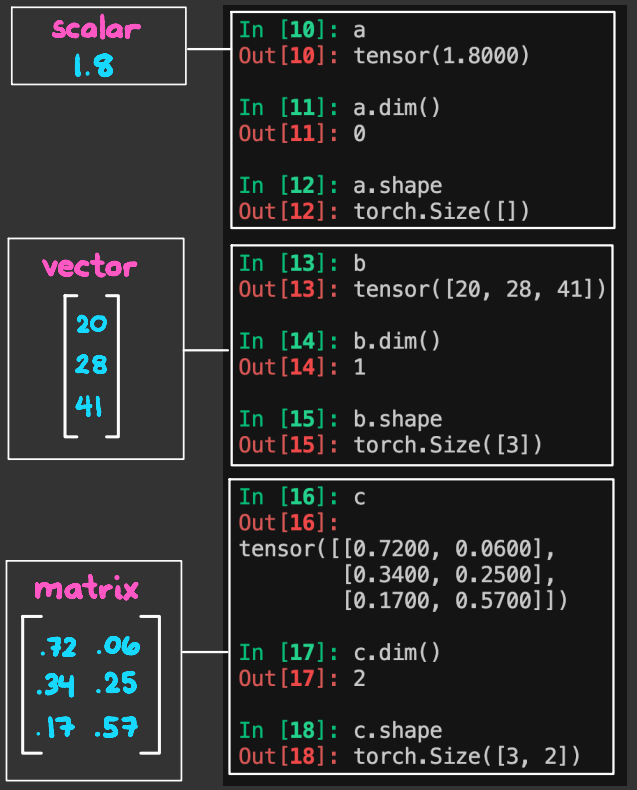
```
In [1]: import torch

In [2]: t = torch.tensor([[1, 2, 3], [4, 5, 6]])

In [3]: t
Out[3]:
tensor([[1, 2, 3],
        [4, 5, 6]])

In [4]: u = torch.zeros_like(t)

In [5]: u
Out[5]:
tensor([[0, 0, 0],
        [0, 0, 0]])
```

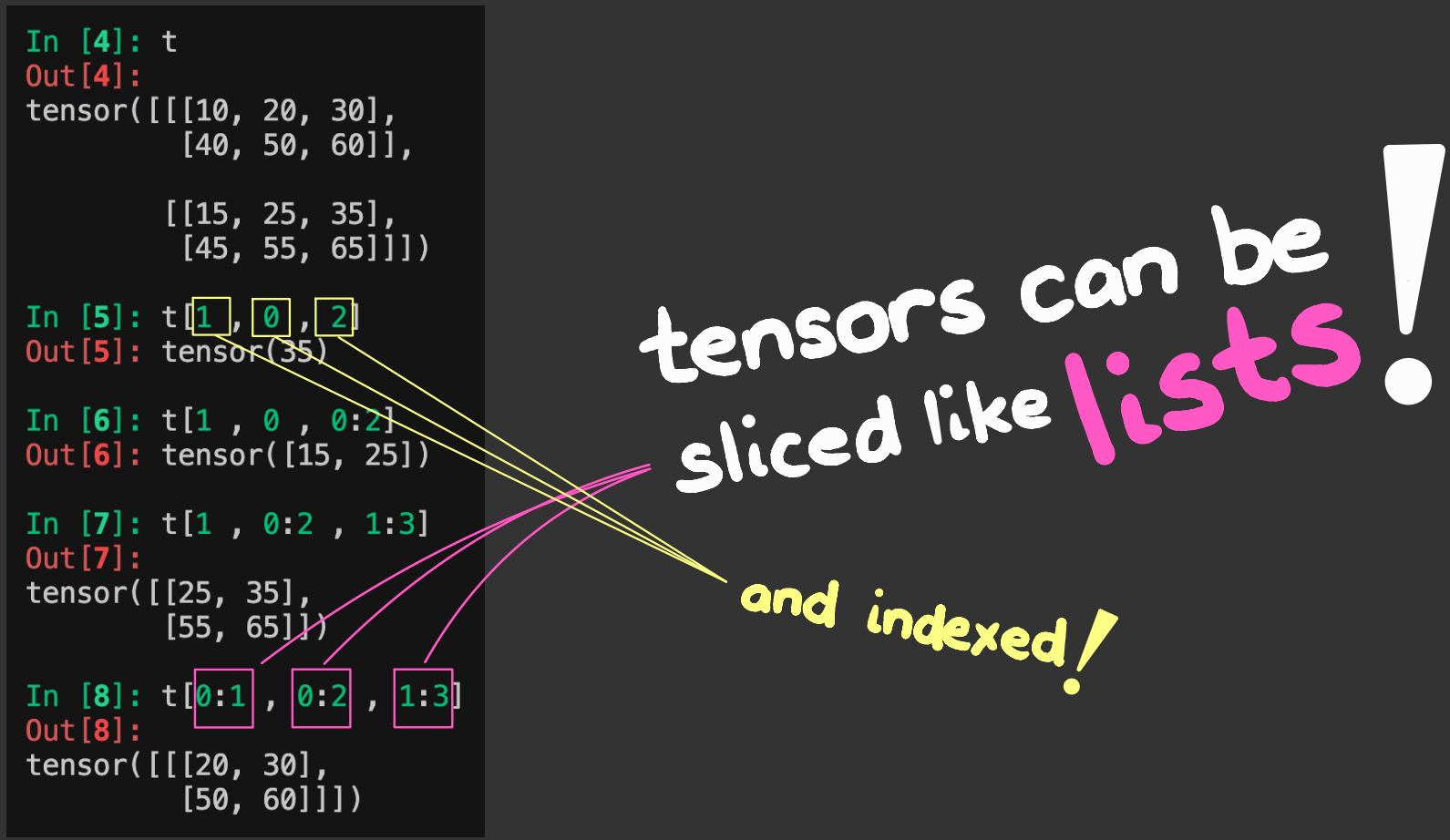


unlike python lists, all elements of a tensor must be the same type

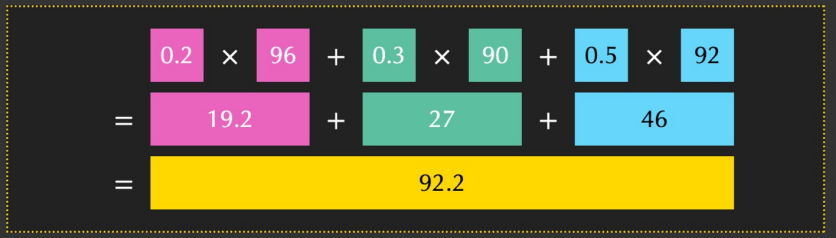


we can ask for the dimension and the shape




In [4]: t
Out[4]:
tensor([[[10, 20, 30],
[40, 50, 60]],
[[15, 25, 35],
[45, 55, 65]]])
In [5]: t[1, 0, 2]
Out[5]: tensor(35)
In [6]: t[1 , 0 , 0:2]
Out[6]: tensor([15, 25])
In [7]: t[1 , 0:2 , 1:3]
Out[7]:
tensor([[25, 35],
[55, 65]])
In [8]: t[0:1 , 0:2 , 1:3]
Out[8]:
tensor([[[20, 30],
[50, 60]]])
tensors can be sliced like lists!
and indexed!

we can perform matrix multiplication



0.2	0.3	0.5
0.2	0	0.8

A

80	90	96
95	80	90
90	70	92

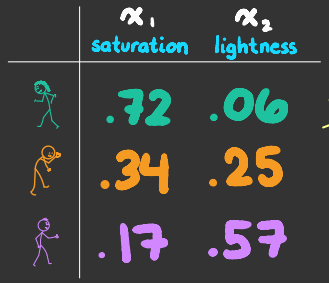
B

89.5	77	92.2
88	74	92.8

AB

```
In [4]: a
Out[4]:
tensor([[0.2000, 0.3000, 0.5000],
        [0.2000, 0.0000, 0.8000]])

In [5]: b
Out[5]:
tensor([[80., 90., 86.],
        [95., 80., 90.],
        [90., 70., 92.]])

In [6]: a.shape
Out[6]: torch.Size([2, 3])

In [7]: b.shape
Out[7]: torch.Size([3, 3])

In [8]: a @ b
Out[8]:
tensor([[89.5000, 77.0000, 90.2000],
        [88.0000, 74.0000, 90.8000]])
```

but we have to make sure they have the right shapes



rows and cols

```
In [6]: x
Out[6]:
tensor([[0.7200, 0.0600],
        [0.3400, 0.2500],
        [0.1700, 0.5700]])

In [7]: x.shape
Out[7]: torch.Size([3, 2])

In [8]: x = x.transpose(0, 1)

In [9]: x
Out[9]:
tensor([[0.7200, 0.3400, 0.1700],
        [0.0600, 0.2500, 0.5700]])

In [10]: x.shape
Out[10]: torch.Size([2, 3])

In [11]: theta
Out[11]: tensor([ 2., -1.])

In [12]: theta.shape
Out[12]: torch.Size([2])

In [13]: theta = theta.unsqueeze(dim=0)

In [14]: theta
Out[14]: tensor([[ 2., -1.]])

In [15]: theta.shape
Out[15]: torch.Size([1, 2])

In [16]: theta @ x
Out[16]: tensor([[ 1.3800,  0.4300, -0.2300]])
```

transposed

$$\begin{bmatrix} .72 & .34 & .17 \\ .06 & .25 & .57 \end{bmatrix}$$

$\theta = \begin{pmatrix} 2.0 & -1.0 \end{pmatrix}$

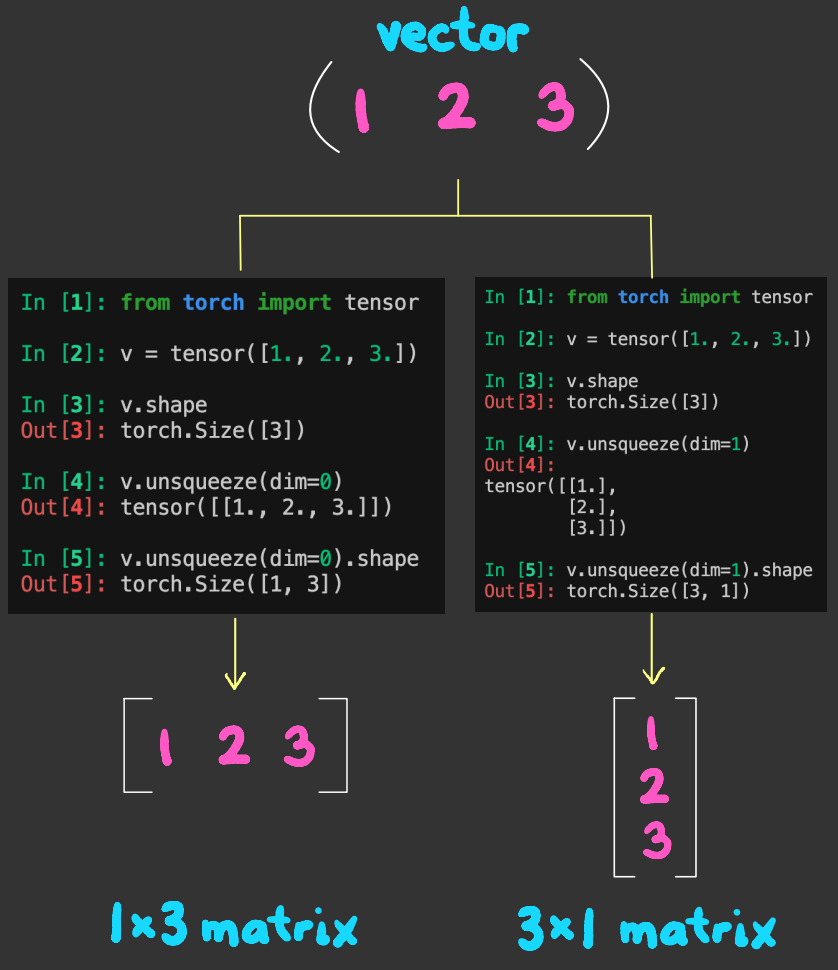
from a vector

to a matrix

$$\begin{bmatrix} 2.0 & -1.0 \end{bmatrix}$$

$$\begin{bmatrix} 2.0 & -1.0 \end{bmatrix} \times \begin{bmatrix} .72 & .34 & .17 \\ .06 & .25 & .57 \end{bmatrix} = \begin{bmatrix} 1.38 & .43 & -.23 \end{bmatrix}$$

unsqueeze converts vectors to matrices



squeeze converts matrices to vectors (when possible)

```
In [1]: from torch import tensor

In [2]: v = tensor([[1., 2., 3.]])

In [3]: v.shape
Out[3]: torch.Size([1, 3])

In [4]: v.squeeze()
Out[4]: tensor([1., 2., 3.])

In [5]: v.squeeze().shape
Out[5]: torch.Size([3])
```

pointwise operations

apply the same operator to every element of a tensor

```
In [3]: x
Out[3]:
tensor([[ 1.3000, -1.2000, -2.0000],
        [ 3.4000,  5.0000, -4.0000]])

In [4]: x > 0
Out[4]:
tensor([[ True, False, False],
        [ True,  True, False]])

In [5]: x.abs()
Out[5]:
tensor([[1.3000, 1.2000, 2.0000],
        [3.4000, 5.0000, 4.0000]])

In [6]: x + 100
Out[6]:
tensor([[101.3000,  98.8000,  98.0000],
        [103.4000, 105.0000,  96.0000]])

In [7]: x.sqrt()
Out[7]:
tensor([[1.1402,    nan,    nan],
        [1.8439, 2.2361,    nan]])

In [8]: x.clamp(min=0)
Out[8]:
tensor([[1.3000, 0.0000, 0.0000],
        [3.4000, 5.0000, 0.0000]])
```

reduction operations
reduce a dimension
(or the entire tensor)
to a single value

```
In [3]: x
Out[3]:
tensor([[ 1.3000, -1.2000, -2.0000],
        [ 3.4000,  5.0000, -4.0000]])

In [4]: x.mean()
Out[4]: tensor(0.4167)

In [5]: x.mean(dim=0)
Out[5]: tensor([ 2.3500,  1.9000, -3.0000])

In [6]: x.mean(dim=1)
Out[6]: tensor([-0.6333,  1.4667])

In [7]: x.sum()
Out[7]: tensor(2.5000)

In [8]: x.sum(dim=0)
Out[8]: tensor([ 4.7000,  3.8000, -6.0000])

In [9]: x.sum(dim=1)
Out[9]: tensor([-1.9000,  4.4000])
```